LIN7076 – Foundations of Computational Linguistics

Recurrent Neural Networks

Adèle Hénot-Mortier

01/12/2025

Queen Mary University of London

Plan for today

- Introduce machine translation and its main challenges.
- Show how recurrent neural nets can handle translation.
- Mention some challenges in the training of simple recurrent networks and sketch a more complex architecture.

Machine translation

A simple machine translation algorithm

- Suppose we have a digitized dictionary between language *A* (**original language**) and language *B* (**target language**), pairing words of *A* with words of *B*.
- To translate a sentence in *A* to *B*, we go through each word of *A*, and output *A*'s translation into *B*.
- Issues?

Cross-linguistic variation: word order

- Though there are some strong trends, languages of the world can vary a lot in terms of **word order**.

(1) Jo ate the cake. English, SVO

(2) Jo-wa keeki-o tabeta.
Jo.NOM cake.NOM eat.PST
'Jo ate the cake.' Japanese, SOV

(3) Den Kuchen hat Jo gegessen
The.ACC cake has Jo eaten.
'Jo has eaten the cake.' German, V2

(4) Jo hat den Kuchen gegessen
Jo has The.ACC cake eaten.
'Jo has eaten the cake.' German, V2

(5) D'ith Jo an císte
Eat.PST Jo the cake.
'Jo ate the cake.' Irish, VSO

Cross-linguistic variation: what's a word?

- In some languages, each specific word encode **one specific meaning**; in other languages, a single word may contain a core meaning but also **a lot of extra functional information**.

(6) Causative-Applicative-Passive stacking in Chichewa (Hyman, 2003)

ndodo i-ná-líl-its-il-idw-á ána
stick SBJ-TAM-cry-C-A-P-FV children

'A stick was used to make the children cry.'

(7) German compounds

Donaudampf**schiff**fahrts**gesellschaft**skapitän
Danube-steam-ship-trip-company-captain

'Captain of the Danube steamship shipping company.'

Cross-linguistic variation: context vs. morphology

- Languages vary a lot in terms of what they encode **overtly vs. contextually**.

(8) Tenselessness in Paraguayan Guarani (Pancheva & Zubizarreta, 2023)

a-jahu
1SG-bathe

'I bathe(d).'

(9) Honorificity (Wang, 2025)

Vous êtes une bonne personne
You.HON/PL are a good person.

'You (whom I respect) are a good person.'

(10) No mass-count distinction in classifier languages (Chierchia, 1998)

a. yí lì mǐ
one CL rice

'one (grain of) rice.'

b. liǎng lì mǐ
two CL rice

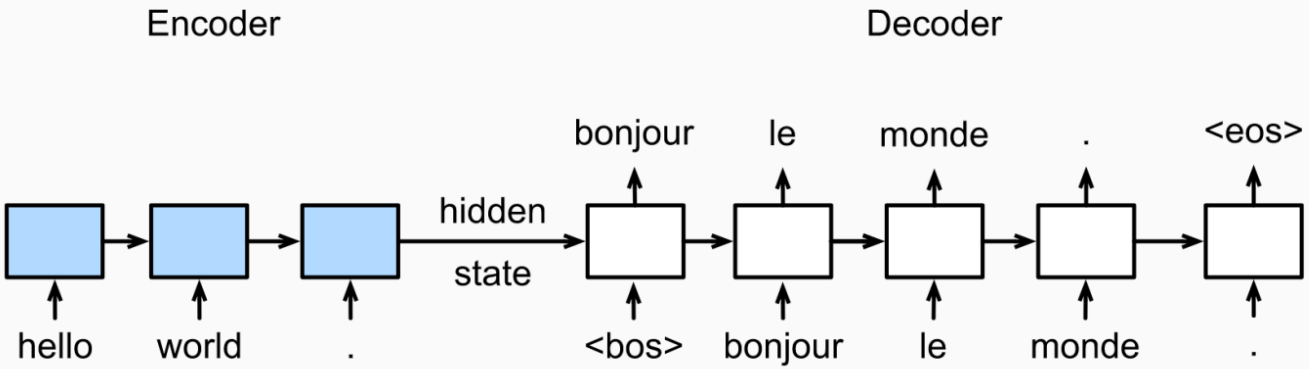
'two (grains of) rice.'

Challenges for machine translation

- We want a system taking a sentence in the original language *A*, and returning its translation in the target language *B*, s.t.:
  - The **syntactic relations** in *A* (subject, object, verb, morphology...) are well-identified and correctly translated in *B*, potentially in a different order, and with more or less words or overt markers.
  - The **meaning** of the sentence and its translation is the same, i.e. the contextual meaning of words is well rendered.
- But we also want the system to be efficient, and go over the original sentence in one pass.

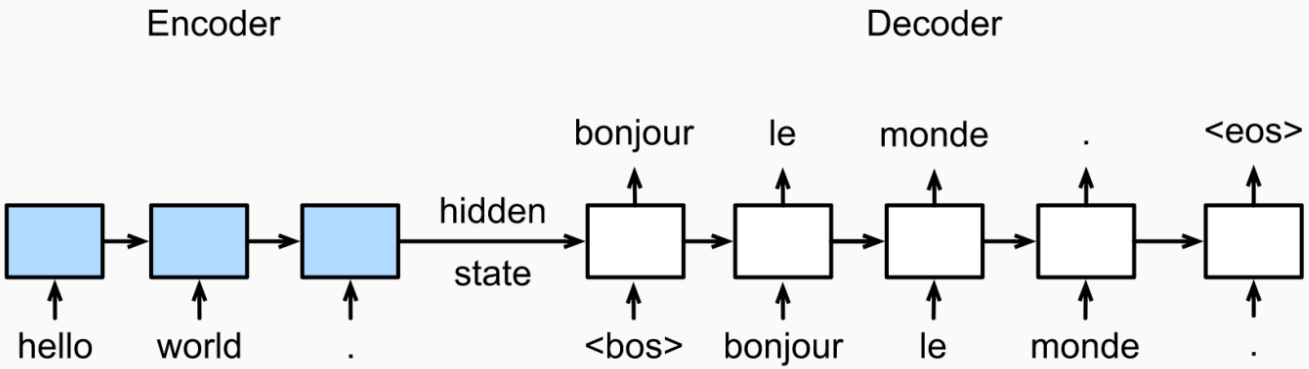
Recurrent neural networks

The sequence-to-sequence model



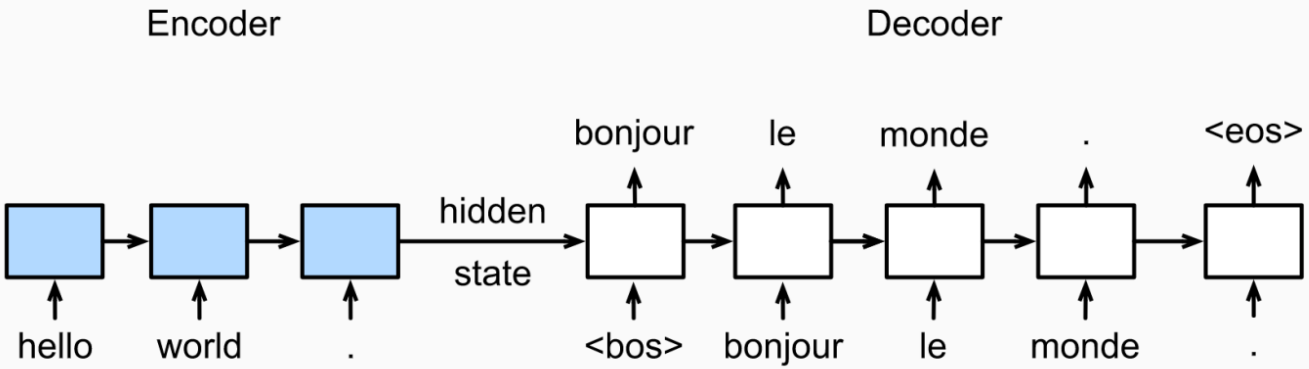
- Feed the model the words in the original language, **one by one**.
- Each word that feeds the model updates its "**hidden state**" (a vector). The hidden state **accumulates information** about the original sentence in a left-to-right fashion.
- Once the sentence is entirely digested, the model starts **spitting out words in the target language based on its hidden state**. It stops when a end-of-sentence token is produced.
- Each word that gets spit out updates the hidden state and thereby influences which word will be produced next.

Architectural advantages of s2s models



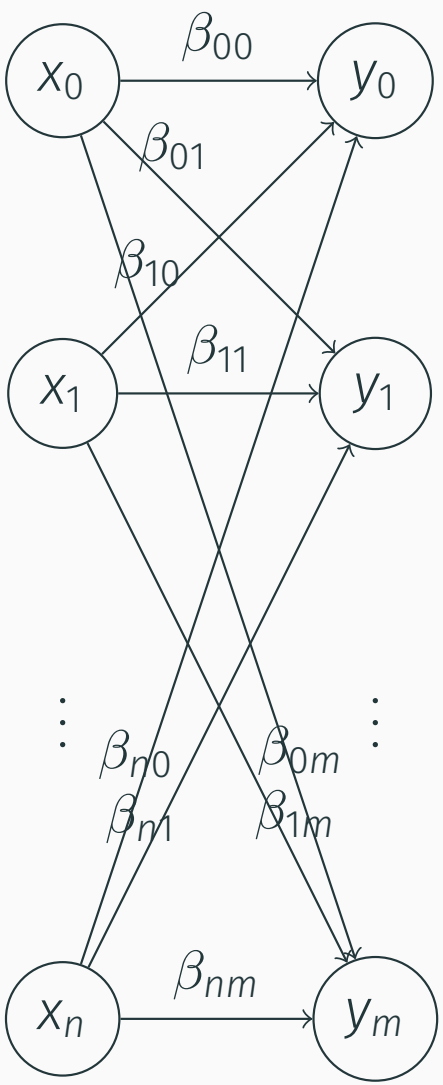
- Waiting until the original sentence is fully processed ensures that the target sentence can follow a different word order.
- The flexible stopping condition ensures that the target sentence can have more or less words than the original sentence.
- The incremental, one-pass computation of the hidden state guarantees efficiency (and enables gradient descent), while encoding contextual information.
- The continued update of the hidden state *while the target sentence is getting produced* makes sure the output is consistent.

An implementation: recurrent neural nets



- Recurrent neural nets (RNNs) are a way to specify which computations take place in the boxes in the above figure.
- A RNN will be specified by 3 linear transformations, and 2 activation functions (non-linearities).

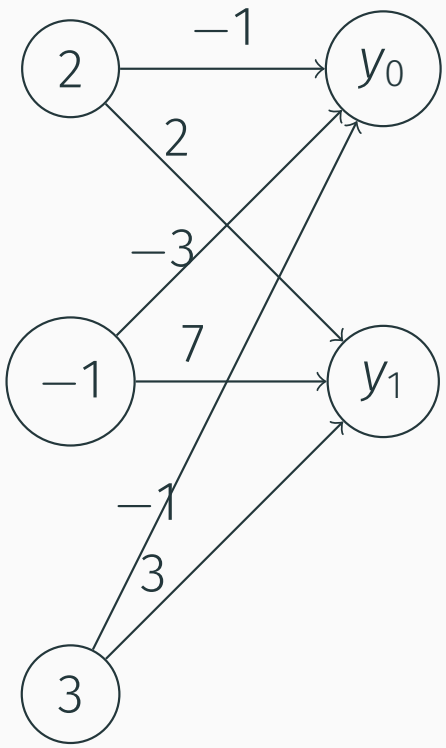
Linear transformation



- Recall that in feedforward neural nets, neurons are connected to each other by weighted arrows indicating multiplication.
- Two fully connected layers of dimensions n and m (without activations) correspond to a linear transformation mapping a n-dimensional vector to a m-dimensional vector.
- This transformation can be "packaged" as a $m \times n$ matrix:

$$\begin{bmatrix} \beta_{00} & \beta_{10} & \dots & \beta_{n0} \\ \beta_{01} & \beta_{11} & \dots & \beta_{n1} \\ \dots & \dots & \dots & \dots \\ \beta_{0m} & \beta_{1m} & \dots & \beta_{nm} \end{bmatrix} \vec{x} = \vec{y}$$

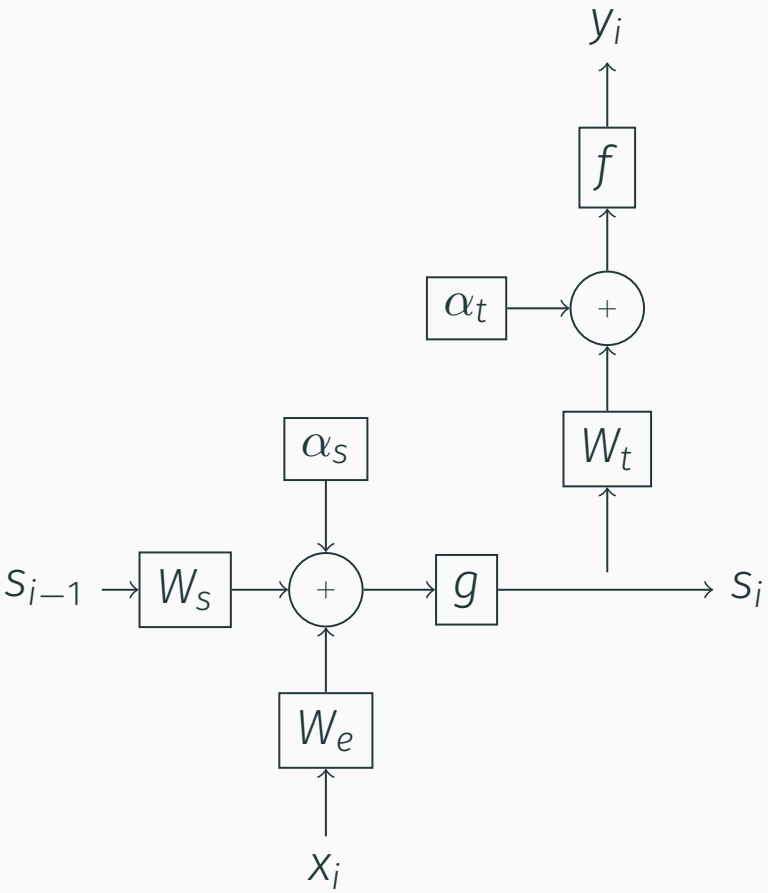
Linear transformation practice



- $y_0 = (-1) \times 2 + (-3) \times (-1) + (-1) \times 3 = -2 + 3 - 3 = -2.$
- $y_1 = 2 \times 2 + 7 \times (-1) + 3 \times 3 = 4 - 7 + 9 = 6.$

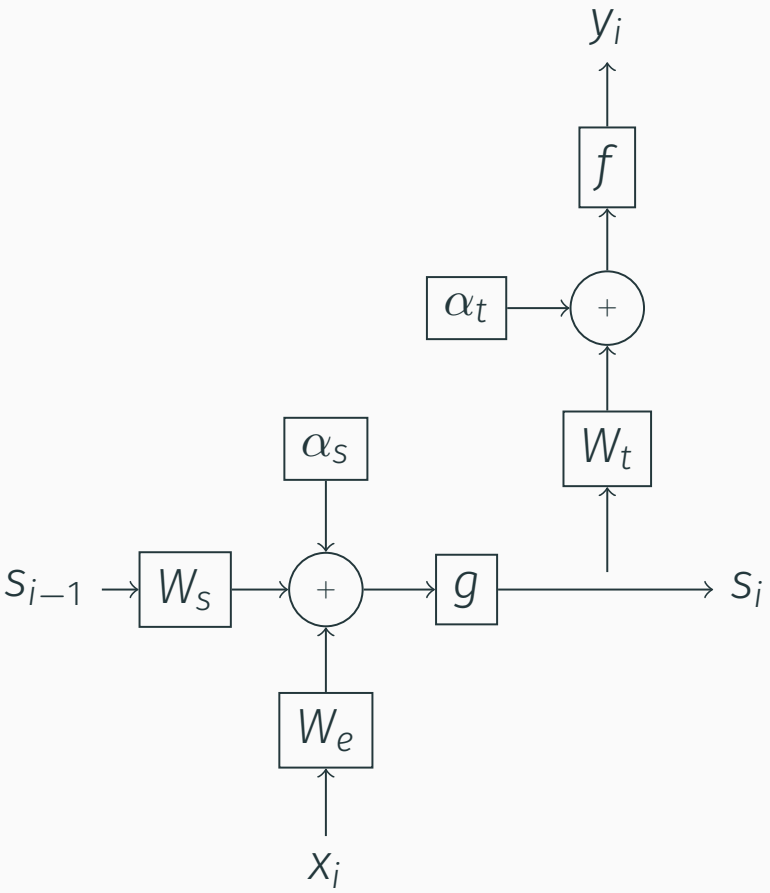
$$\begin{bmatrix} -1 & -3 & -1 \\ 2 & 7 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix} = \begin{bmatrix} -2 \\ 6 \end{bmatrix}$$

RNN unit



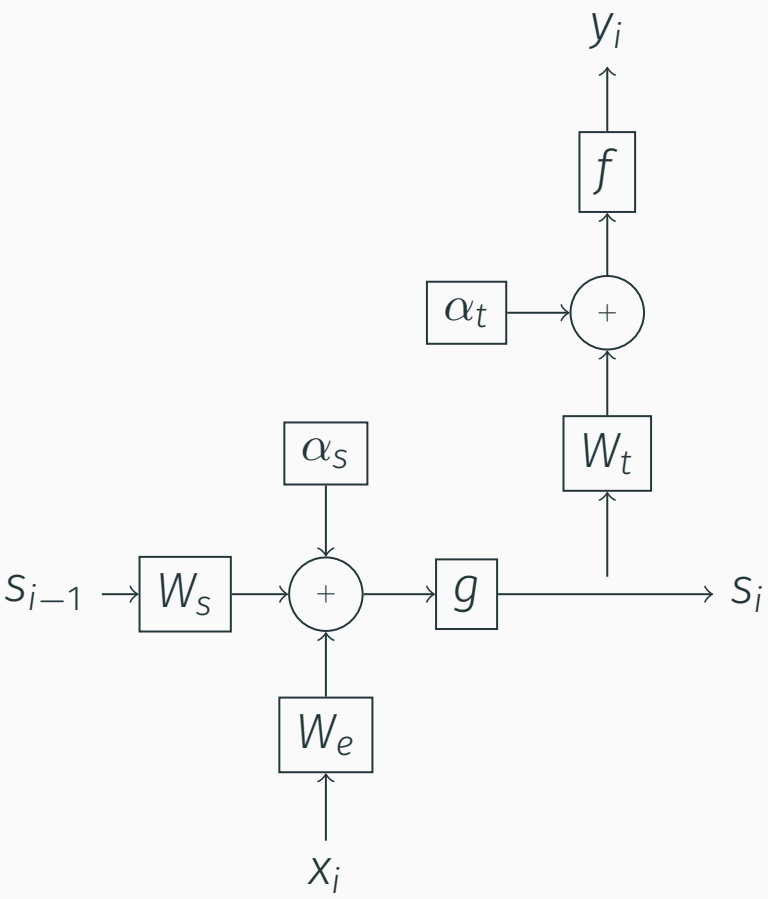
- x_i encodes i-th word of the input sequence; y_i encodes the i-th word of the output sequence.
- Words are encoded as one-hot vectors: vectors with 0s everywhere, except at the component which corresponds to the word they encode. This components is 1.
- s_{i-1} and s_i are states, that build up information about the sentence.

RNN unit



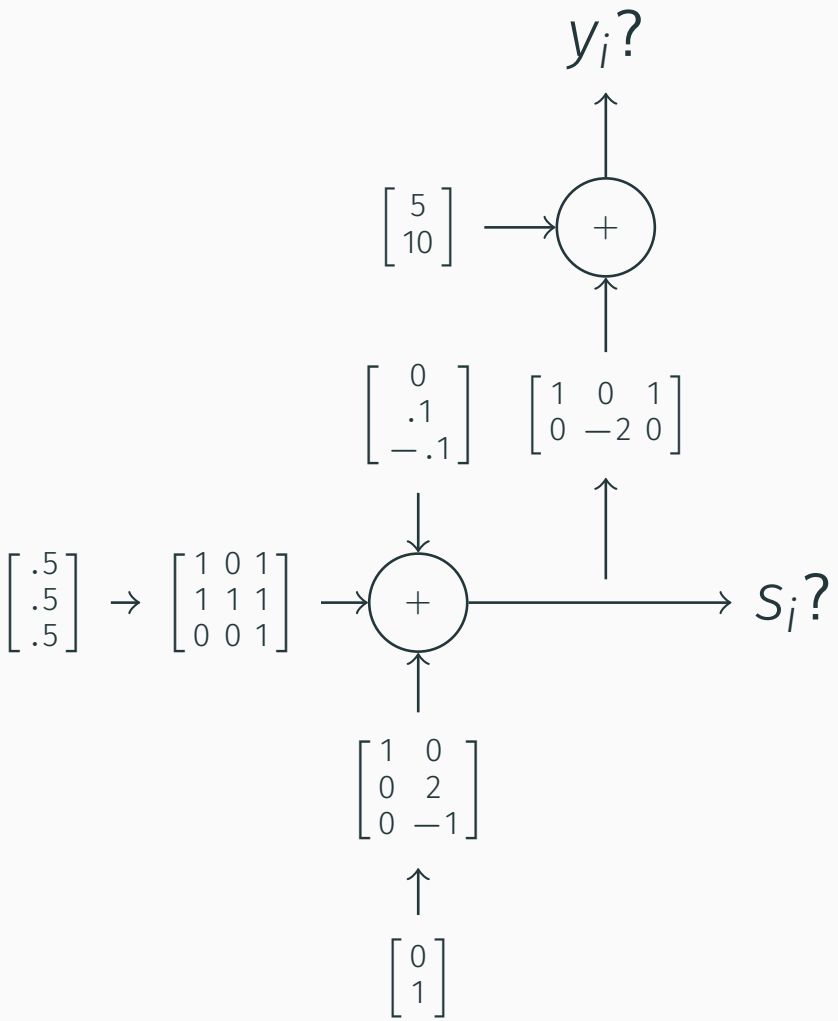
- W_e, W_s and W_t are linear transformations, i.e. matrices.
- W_e maps the one-hot vector x_i into a denser, more meaningful vector representation (think a static word embedding). The dimension of that vector matches that of the state.
- W_s is a transformation over states; it does not change dimensionality.
- W_t maps a state to an output vector; it may change dimensionality.

RNN unit



- α_s and α_t are fixed vector biases/intercepts. α_s as the same dimension as state vectors; α_s as the same dimension as output vectors.
- f and g are activation functions.
- In the case of machine translation, g will often be a hyperbolic tangent (*tanh*)–think sigmoid, but between -1 and 1.
- f will be a softmax, which can be seen as the generalization of a sigmoid when there are more than two classes.

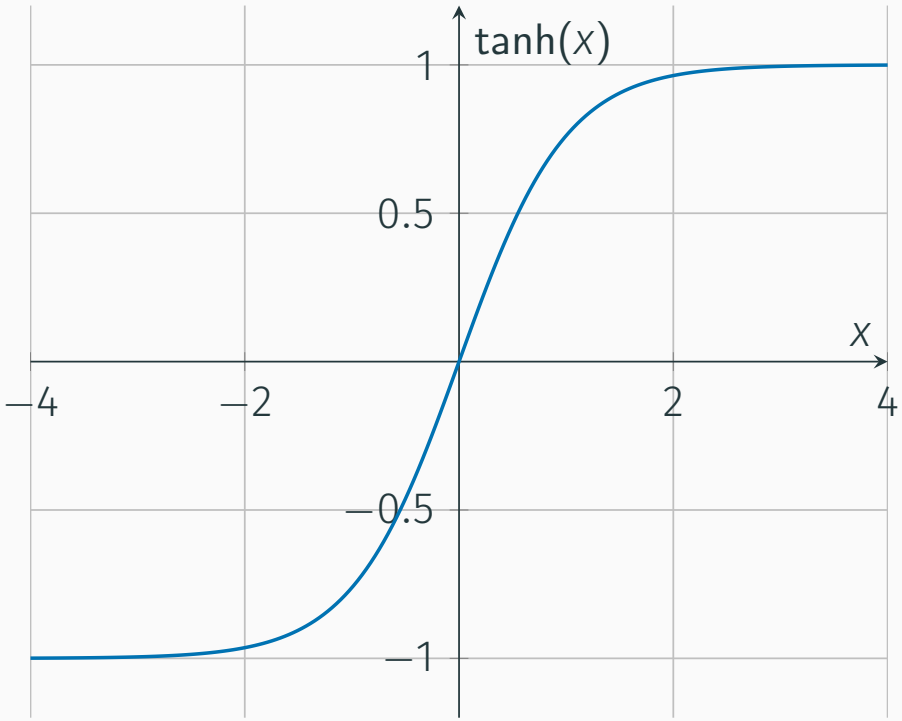
RNN practice, no activation



- $\begin{bmatrix}1&0&1\\1&1&1\\0&0&1\end{bmatrix}\begin{bmatrix}.5\\.5\\.5\end{bmatrix}=\begin{bmatrix}1\\1.5\\.5\end{bmatrix}$
- $\begin{bmatrix}1&0\\0&2\\0&-1\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix}=\begin{bmatrix}0\\2\\-1\end{bmatrix}$
- $\begin{bmatrix}1\\1.5\\.5\end{bmatrix}+\begin{bmatrix}0\\2\\-1\end{bmatrix}+\begin{bmatrix}0\\.1\\-.1\end{bmatrix}=\begin{bmatrix}1\\3.6\\-.6\end{bmatrix}=s_i$
- $\begin{bmatrix}1&0&1\\0&-2&0\end{bmatrix}\begin{bmatrix}1\\3.6\\-.6\end{bmatrix}=\begin{bmatrix}.4\\-7.2\end{bmatrix}$
- $\begin{bmatrix}.4\\-7.2\end{bmatrix}+\begin{bmatrix}5\\10\end{bmatrix}=\begin{bmatrix}5.4\\2.8\end{bmatrix}=y_i$

The thanh activation function

- Shaped like a sigmoid, but bounded by -1 and 1 instead of 0 and 1.
- Nice slope, differentiable, works with gradient descent.
- Used at the level of hidden states (g). If applied to a vector, applies element-wise.

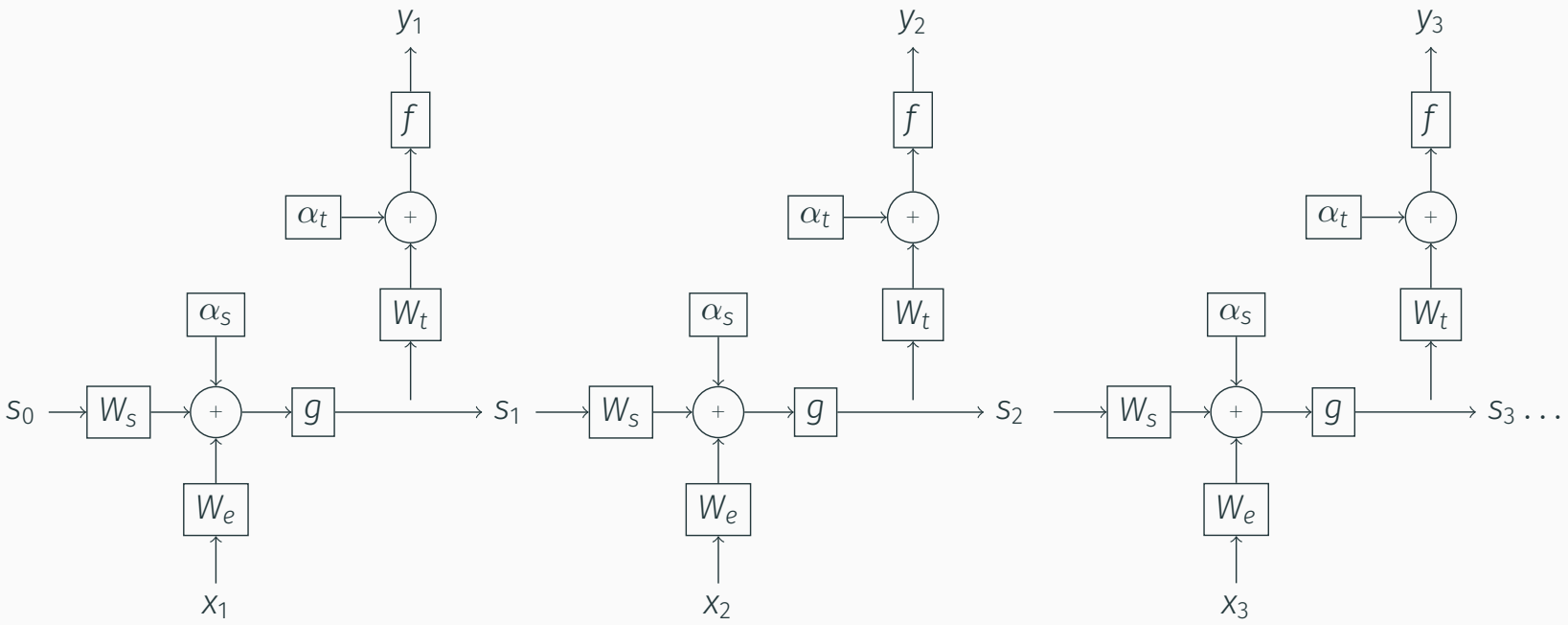


The softmax activation function

- Takes a vector and “exponentially” rescales its component to return a vector with same dimension, but whose components are each in the interval $[0; 1]$ and add up to 1. These components can together be understood as a probability distribution.
- Softmax is used for multinomial classification, i.e. when one wants to predict one class out of $n > 2$ classes. For instance, when one wants to predict one word, among n words in the lexicon.
- For that reason, we use softmax as f in machine translation RNNs, to predict target words.

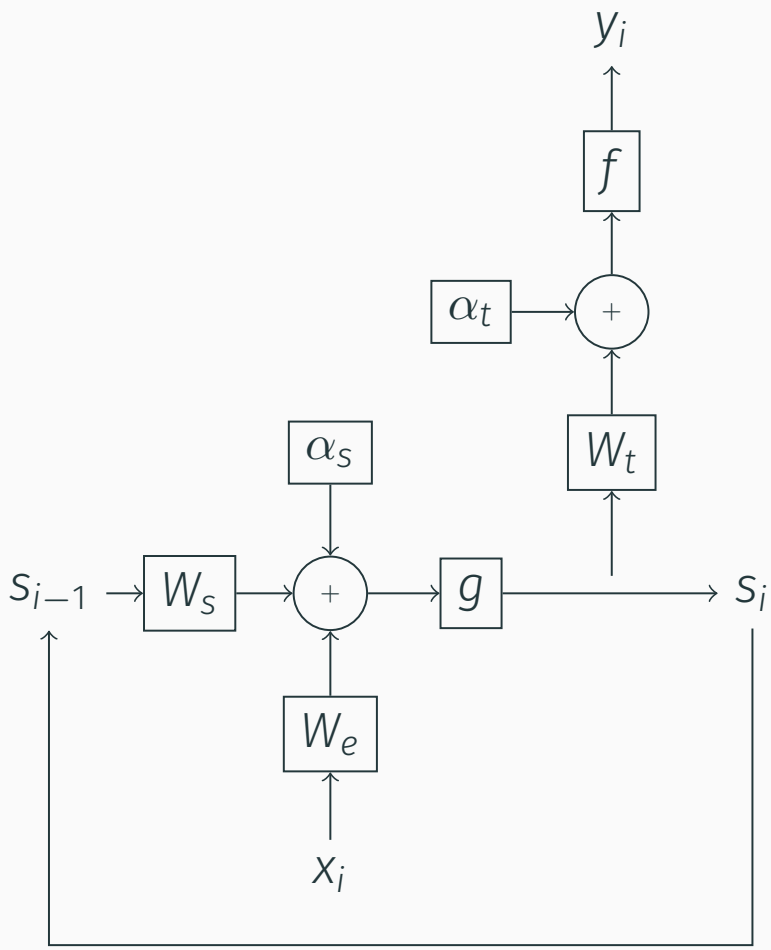
$$softmax\left(\begin{bmatrix} 1 \\ -3 \\ 2 \end{bmatrix}\right) = \begin{bmatrix} \frac{e^1}{e^1+e^{-3}+e^2} \\ \frac{e^{-3}}{e^1+e^{-3}+e^2} \\ \frac{e^2}{e^1+e^{-3}+e^2} \end{bmatrix} = \begin{bmatrix} 0.268 \\ 0.005 \\ 0.7275 \end{bmatrix}$$

Why “recurrent”?



- Each input word x_i influences the states s_j/the outputs y_j for $j \geq i$.
- Linguistic information can “flow” left-to-right, but may end up “fading” after a few units.

Alternative representation (that I don't like!)



- Many NLP resources schematize the stacking of RNNs units using a feedback loop.

Preparing data for machine translation

- For machine translation, the model takes an input sentence in the original language and returns an output sentence in the target language.
- The input and output sequences have the following form:

Output	Empty tokens	Target sentence		</s>
Input	**Original sentence**	<s>	**Target sentence**	Empty token

- The model learns to reproduce this pattern.
- This pattern can also be used for tasks such as question-answering: in that case, the original sentence is a question, and the target sentence is the answer.

Preparing data for language generation

- For language generation, the model takes an a “seed” sentence (or part of a sentence) and iteratively produces continuations of the sentence.
- The model is trained to reproduce the input sentence from a prefix of the sentence of fixed length (i.e. the first 10 words).
- If the prefix is just 1 word, the input and output sequences have the following form:

Output		w_1	w_2	...	</s>
Input	<s>	w_0	w_1	...	w_n

- If the prefix if of size k, the sequences look like this:

Output					wk	wk+1	...	</s>
Input	<s>	w_0	w_1	...	w_{k-1}	w_k	...	w_n

LSTMs

Optimizing RNNs

- By “unrolling” a RNN out to model a particular sequence, we can treat the whole thing as a feedforward network with a lot of parameter sharing
- Thus, we can **tune the parameters using gradient descent**, and learn to model sequential mappings!
- However, the gradient that gets propagated throughout the network is a product of gradients,[1] and this product will either grow or shrink exponentially with the length of the sequence, and make it very difficult to train...
- This issue is known as **vanishing gradients**.

[1] If the output at the end of the sequence is incorrect, and depends critically, via the weights, on the input at time 1, the gradients has the form $\frac{\delta s_1}{\delta s_1}\frac{\delta s_3}{\delta s_2}\ldots\frac{\delta s_n}{\delta s_{n-1}}$

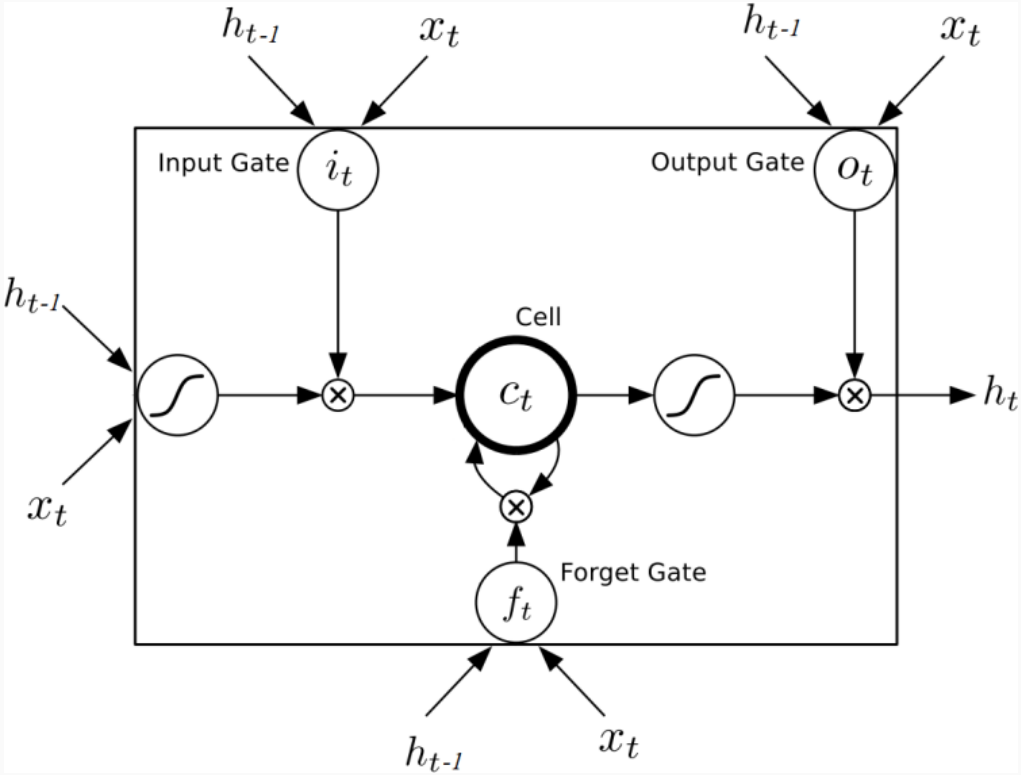
Gates to the rescue

- RNNs update their entire hidden state at each cycle, which causes the gradients to vanish/explode and the training to be difficult.
- We'd like a similar model that would only update some parts of its memory on each cycle.
- Such a model would be more able to retain state values over time and its gradients better-behaved.
- This can be achieved *via* **gating**.

LSTMs

- LSTM stands for “long short-term memory.”
- LSTMs have a memory cell (=our state vector in RNNs) but also three **gating networks**.
  - The **input gate** selects which dimensions of the state will be updated with new values;
  - the **forget gate** decides which dimensions of the state will have its old values moved toward 0;
  - The **output gate** decides which dimensions of the state will be used to compute the output value.

LSTM diagram and equations



$$i_t = \sigma(U^i x_t + W^i h_{t-1})$$
$$o_t = \sigma(U^o x_t + W^o h_{t-1})$$
$$f_t = \sigma(U^f x_t + W^f h_{t-1})$$
$$\tilde{c}_t = tanh(U^g x_t + W^g h_{t-1})$$
$$c_t = \sigma(f_t c_{t-1} + i_t \tilde{c}_t)$$
$$h_t = tanh(c_t) o_t$$

Lab

https://colab.research.google.com/drive/1Tj8hY08CWO5bJKod2mA7gl5LVcTJu-Fux?usp=sharing